

DAY 17

Embeddings and Vector Search in Depth

GPT + AI Agents + RAG + n8n + API Integration + Business Automation

Duration	60 minutes
Level	Beginner to Intermediate
Main Project	Semantic Company Knowledge Search Engine
Core Transition	From "documents become chunks" to "machines can search meaning mathematically"

Day 17 Core Question

How can a computer find text that is **similar in meaning** even when the user and the document use different words?

Prepared for the 30-Day AI Automation Lecture Series

Table of Contents

1. Learning Goals & Schedule
2. Day 16 Review
3. What Is an Embedding?
4. Vector Space Intuition
5. Similarity Metrics
6. Semantic Search
7. Vector Indexing & Nearest Neighbors
8. Vector Stores & Metadata
9. Retrieval Controls: Top-K, Thresholds, Ranking
10. Hybrid Search & Query Rewriting
11. Embeddings Inside RAG
12. OpenAI Embeddings & Vector Search
13. Practical Python Mini Project
14. Evaluation & Failure Modes
15. Business Use Cases
16. Exercise, Quiz & Homework
17. Interview Questions & Day 18 Preview
18. References

Teaching Note

Day 17 is intentionally more mathematical than earlier lectures, but the mathematics stays practical. Students only need enough vector intuition to design and debug semantic search systems.

1. Learning Goals & 60-Minute Schedule

Day 17 turns the RAG pipeline from Day 16 into a concrete retrieval mechanism. Students learn how text becomes vectors, how similarity is computed, and how a vector search engine chooses candidate chunks.

Learning Objectives

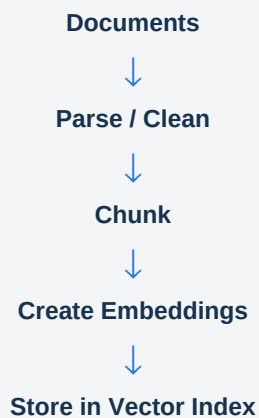
- Explain embeddings in plain business language.
- Describe a vector as a list of numbers representing semantic features.
- Explain why similar meanings tend to have nearby vector representations.
- Understand cosine similarity, dot product, and Euclidean distance conceptually.
- Explain semantic search versus keyword search.
- Understand exact nearest-neighbor versus approximate nearest-neighbor search.
- Explain the purpose of a vector store and vector index.
- Use metadata filters, top-K retrieval, score thresholds, and ranking controls.
- Explain how embeddings, chunks, vector stores, and an LLM combine inside RAG.
- Build and test a small semantic search prototype.

Time	Topic
0-5	Day 16 review
5-12	What embeddings are
12-20	Vector space and similarity
20-28	Semantic search
28-35	Vector indexes and nearest neighbors
35-42	Metadata, top-K, thresholds, ranking
42-48	Hybrid search and query rewriting
48-54	OpenAI embeddings + vector search
54-58	Mini project
58-60	Quiz + homework

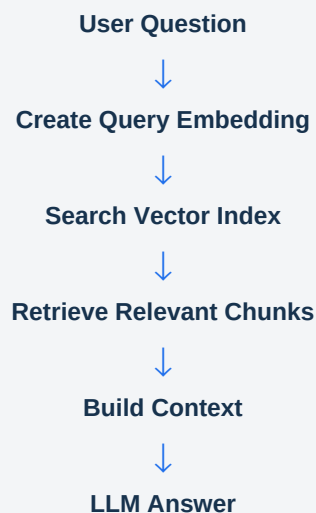
2. Day 16 Review: Where Embeddings Fit

Day 16 introduced two separate RAG pipelines: an **indexing pipeline** and a **query pipeline**.

Indexing pipeline



Query pipeline



Key Idea

The same embedding model maps both stored chunks and new user queries into the same vector space. Search then compares the query vector with stored vectors.

Three Different Jobs

Component	Job	Example
Embedding model	Convert meaning into numbers	"remote work policy" -> vector
Vector index/store	Find nearby vectors efficiently	Return the 5 most similar chunks
LLM	Read retrieved text and answer	Explain policy using retrieved evidence

3. What Is an Embedding?

An **embedding** is a numerical representation of an item - such as a sentence, paragraph, product, or image - designed so that related items have useful geometric relationships.

Beginner Definition

An embedding converts **meaning into coordinates**. Instead of storing only words, the system stores a long list of numbers that can be compared mathematically.

Example Concept

Text A: "Employees can work from home two days each week."
Text B: "What is our remote work policy?"
Text C: "The cafeteria closes at 6 PM."

Embedding model:

```
A -> [ 0.12, -0.44, 0.88, ... ]  
B -> [ 0.10, -0.40, 0.91, ... ]  
C -> [-0.63, 0.11, 0.07, ... ]
```

A and B should be semantically closer than A and C.

The individual numbers are usually not human-interpretable labels like "remote-work = 0.88." Meaning is distributed across many dimensions.

What Embeddings Are Good For

- Semantic search
- Recommendation systems
- Clustering
- Classification by similarity
- Duplicate detection
- Anomaly detection
- RAG retrieval

4. Vector Space Intuition

A vector is simply an ordered list of numbers. In teaching examples we can draw vectors in 2D or 3D. Real embedding vectors usually have far more dimensions.

2D Teaching Example

```
Policy A = (1.00, 0.00)
Query B  = (0.80, 0.20)
Lunch C  = (-0.20, 0.95)
```

A and B point in nearly the same direction.
A and C point in very different directions.

Important

Do not interpret real embedding dimensions independently. The useful property is the **relationship between whole vectors**, not the meaning of one coordinate.

Why Many Dimensions?

Language contains many overlapping patterns: topic, intent, entities, sentiment, domain, syntax, relationships, and more. A high-dimensional representation gives the model room to encode these patterns.

Dimensions vs Quality

More dimensions can preserve more representational capacity, but they also increase memory, storage, and search computation. Some embedding APIs allow reduced dimensions for this tradeoff.

5. Similarity Metrics

After text becomes vectors, the retrieval system needs a rule for deciding which vectors are closest.

Cosine Similarity

```
cosine_similarity(a, b) = dot(a, b) / (norm(a) * norm(b))
```

Interpretation for normalized text vectors:

closer to 1 -> more similar direction

around 0 -> weak directional similarity

negative -> opposite direction (depending on embedding space)

For the teaching vectors $A=(1,0)$ and $B=(0.8,0.2)$, cosine similarity is about 0.97 - very high.

Dot Product

The dot product multiplies corresponding dimensions and sums them. When vectors are normalized, dot product and cosine similarity produce equivalent ranking behavior.

Euclidean Distance

Euclidean distance measures straight-line distance between points. Smaller distance means closer vectors.

Metric	Higher or Lower = Better?	Useful intuition
Cosine similarity	Higher	Same direction / semantic orientation
Dot product	Higher	Alignment; often efficient in vector systems
Euclidean distance	Lower	Physical distance between points

Practical Rule

Use the similarity/distance convention expected by your embedding model and vector database. Do not compare score numbers from unrelated systems as if they were universal.

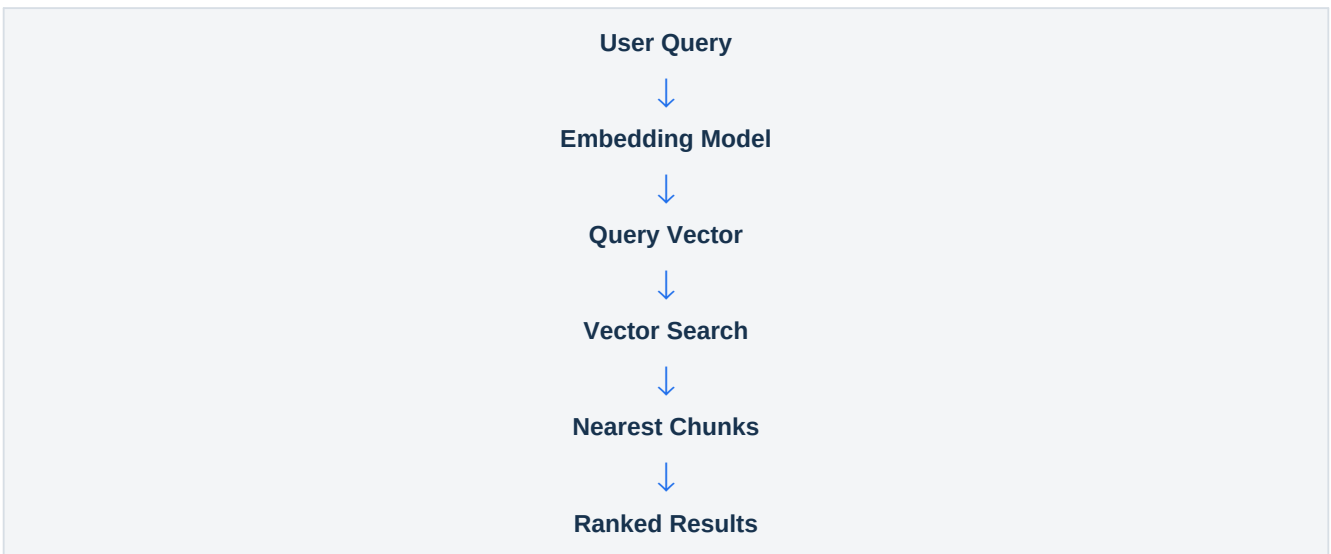
6. Semantic Search

Keyword search asks: **Which documents contain these words?** Semantic search asks: **Which documents mean something similar to this query?**

Example

User Query	Document Text	Keyword Match	Semantic Match
Can I work from home?	Employees may work remotely two days per week.	Weak	Strong
How do I get reimbursed?	Submit expense claims within 30 days.	Weak	Strong
Reset password	Password reset instructions	Strong	Strong

Semantic Search Pipeline



Why This Matters for RAG

Users rarely phrase questions exactly like the source document. Semantic retrieval helps bridge vocabulary differences, paraphrases, and natural language variation.

Example

Query: **“Can I work from another city?”**

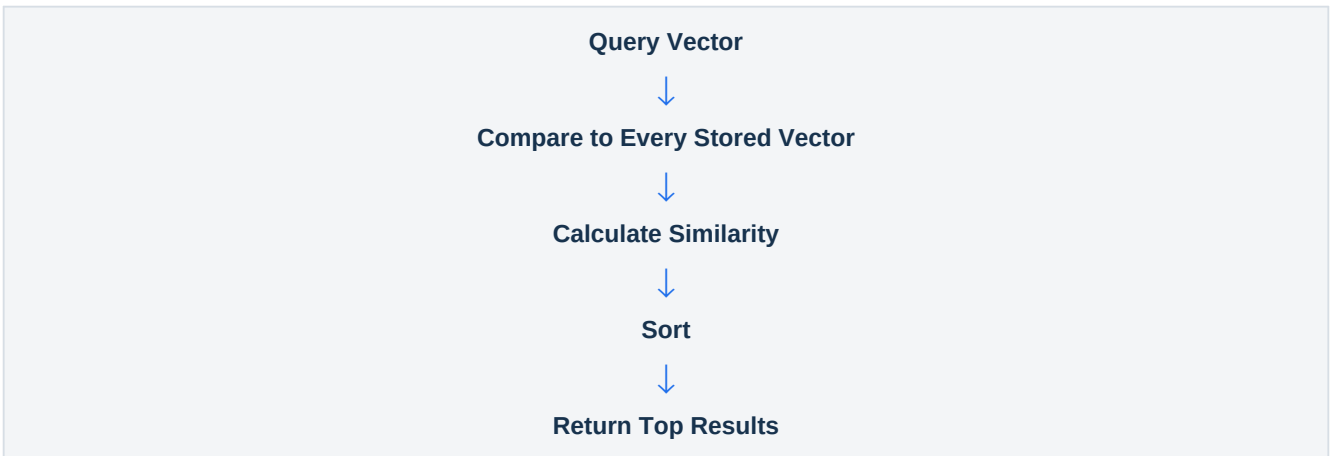
Document: **“Remote employees may temporarily work outside their home office region with manager approval.”**

Keyword overlap may be limited, but semantic retrieval can still surface the policy.

7. Vector Indexing & Nearest Neighbors

If you only have 20 vectors, you can compare the query to every vector. If you have millions of vectors, comparing every vector can become too expensive. Vector indexes are designed to make nearest-neighbor search fast.

Exact Search



Advantage: exact ranking. **Disadvantage:** expensive at large scale.

Approximate Nearest Neighbor (ANN)

ANN methods trade a small amount of exactness for major speed improvements. Instead of evaluating every possible vector, the index navigates a structured search space to find likely nearest neighbors.

HNSW - Concept Only

Hierarchical Navigable Small World (HNSW) is a common ANN indexing idea. Think of it as a graph that lets the search “jump” quickly toward promising neighborhoods rather than scanning every item.

Tradeoff

Goal	Exact Search	ANN Index
Recall	Potentially exact	Usually very high but approximate
Latency at large scale	Higher	Lower
Index complexity	Low conceptually	Higher
Best fit	Small datasets / benchmarks	Large production retrieval

8. Vector Stores & Metadata

A vector store manages vectorized content and the information needed to retrieve it. In RAG, each vector should remain connected to the text chunk and useful metadata.

Example Stored Record

```
{
  "chunk_id": "policy_2026_0042",
  "text": "Employees may work remotely two days per week...",
  "embedding": [0.12, -0.44, 0.88, ...],
  "metadata": {
    "department": "HR",
    "country": "US",
    "document": "employee_handbook.pdf",
    "year": 2026
  }
}
```

Why Metadata Matters

Semantic similarity alone may retrieve a semantically similar but unauthorized, outdated, or wrong-region document. Metadata filtering narrows the candidate set before or during search.

Example Filter

Query: "What is my parental leave policy?"

Filter:
department = "HR"
country = "US"
year >= 2026

Security Rule

Authorization filters must be enforced by trusted application logic. Do not rely on the LLM to “remember” not to retrieve documents the current user is not allowed to access.

9. Retrieval Controls: Top-K, Thresholds, Ranking

Top-K

Top-K means return the K highest-ranked candidates. For example, top-5 retrieval returns the five most relevant chunks according to the retrieval system.

Top-K	Potential Benefit	Potential Risk
2	Small, focused context	May miss supporting evidence
5	Good starting balance	May include some noise
20	Higher recall	More tokens, noise, latency

Score Threshold

A score threshold rejects candidates below a minimum relevance level. This is useful when the system should say “I did not find enough evidence” instead of forcing an answer from weak matches.



Ranking and Re-ranking

First-stage retrieval tries to find a good candidate set quickly. A ranking or re-ranking stage can then reorder those candidates using a stronger relevance model or ranking algorithm.

Important

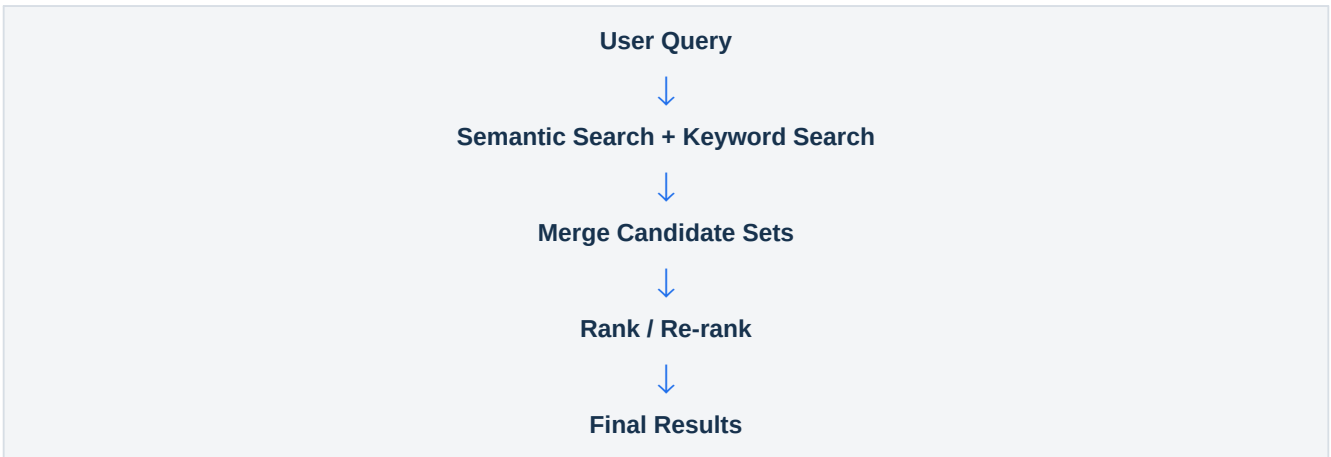
Higher top-K does not automatically mean better answers. Retrieval quality is a precision/recall tradeoff, not a “more context is always better” problem.

10. Hybrid Search & Query Rewriting

Why Semantic Search Alone Can Miss Things

Embeddings are excellent for meaning, but exact tokens can still matter: invoice IDs, product SKUs, error codes, model numbers, legal clauses, names, or unusual abbreviations.

Hybrid Search



Hybrid systems combine semantic similarity with lexical/keyword evidence. This is especially useful for enterprise knowledge where both meaning and exact identifiers matter.

Query Rewriting

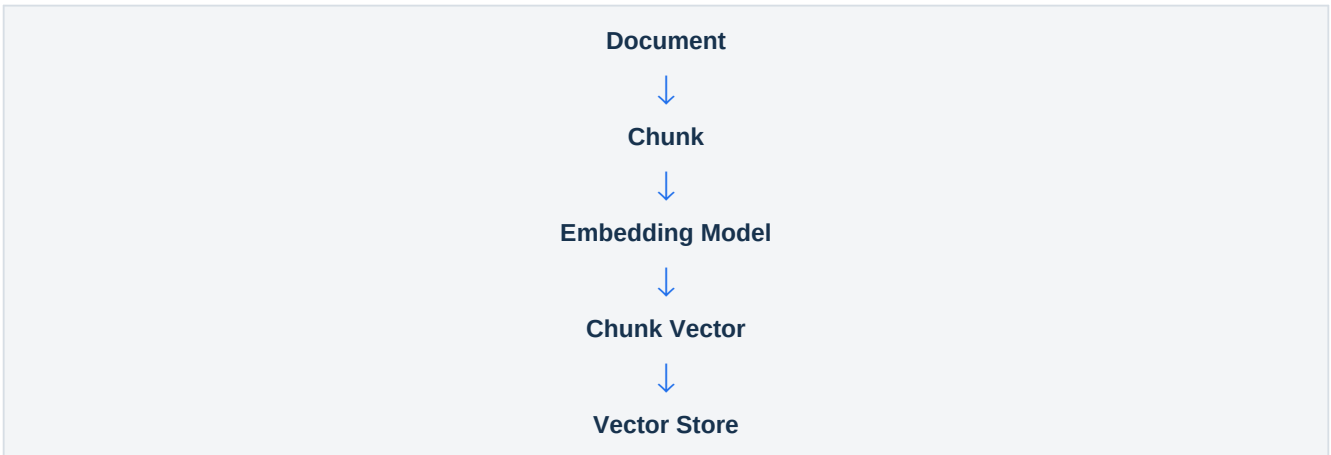
A retrieval system may rewrite or expand the query before search. For example, “PTO carryover” might become “paid time off vacation carry over unused days.” Query rewriting can improve recall but should be tested carefully.

When Query Rewriting Helps

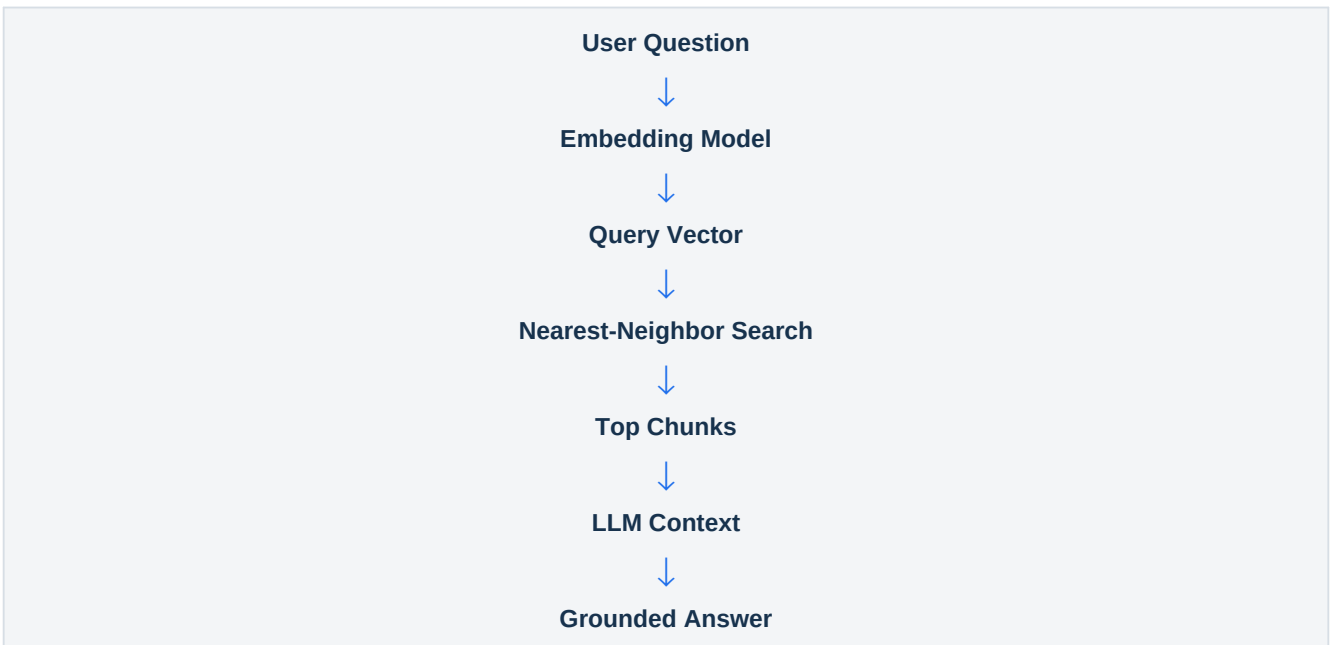
- Very short questions
- Ambiguous acronyms
- User slang versus formal policy language
- Multi-part questions
- Misspellings or incomplete wording

11. Embeddings Inside RAG

Indexing Time



Question Time



Critical Matching Unit

The retriever usually compares the **query embedding** to **chunk embeddings**. This is why chunk quality strongly affects search quality.

Bad Chunk Example

Chunk contains:

- vacation policy
- payroll schedule
- office Wi-Fi instructions
- legal disclaimer
- cafeteria menu

One embedding must represent too many unrelated ideas.

Better

Use chunks that preserve a coherent topic, then retain metadata linking them back to the original file, heading, page, and section.

Retrieval Golden Rule

The LLM cannot ground an answer in evidence that the retriever never found. Retrieval quality is often the first place to debug a weak RAG answer.

12. Current OpenAI Embeddings & Vector Search

The following API-specific notes were verified against official OpenAI documentation on August 18, 2026.

Embedding Models

OpenAI currently documents **text-embedding-3-small** and **text-embedding-3-large** as its third-generation embedding models. The default vector lengths are 1536 and 3072 respectively, and a `dimensions` parameter can reduce the output size.

```
from openai import OpenAI

client = OpenAI()

response = client.embeddings.create(
    model="text-embedding-3-small",
    input="Employees may work remotely two days per week."
)

vector = response.data[0].embedding
print(len(vector))
```

OpenAI Vector Stores

OpenAI Retrieval uses vector stores as searchable indexes for uploaded data. Vector-store search accepts a query and can return relevant chunks with scores.

```
from openai import OpenAI

client = OpenAI()

results = client.vector_stores.search(
    vector_store_id="vs_123",
    query="What is the remote work policy?",
    max_num_results=5,
)

for item in results.data:
    print(item.score, item.filename)
    print(item.content)
```

Current Search Controls

The current vector-store search API documents metadata filters, max result count, ranking options, score thresholds, and optional query rewriting. The current `max_num_results` range is 1 to 50.

Hosted File Search

OpenAI File Search is a hosted Responses API tool that searches uploaded files using semantic and keyword search. It can remove much of the custom retrieval plumbing for applications that fit the hosted workflow.

13. Practical Python Mini Project

Project: **Semantic Company FAQ Search Engine**. We will embed a small FAQ list, embed the user query, compute cosine similarity locally, and return the most similar FAQ entries.

Step 1 - Example Knowledge

```
FAQS = [  
    "Employees may work remotely two days per week.",  
    "Expense reports must be submitted within 30 days.",  
    "Passwords can be reset through the identity portal.",  
    "Unused vacation days may carry over up to 5 days.",  
    "The support team operates Monday through Friday."  
]
```

Step 2 - Create Embeddings

```
from openai import OpenAI  
import numpy as np  
  
client = OpenAI()  
  
resp = client.embeddings.create(  
    model="text-embedding-3-small",  
    input=FAQS  
)  
  
faq_vectors = [np.array(x.embedding) for x in resp.data]
```

Step 3 - Cosine Similarity

```
def cosine_similarity(a, b):  
    return np.dot(a, b) / (  
        np.linalg.norm(a) * np.linalg.norm(b)  
    )
```

Step 4 - Search Function

```
def semantic_search(query, top_k=3):
    q = client.embeddings.create(
        model="text-embedding-3-small",
        input=query
    )
    qv = np.array(q.data[0].embedding)

    scored = []
    for text, vector in zip(FAQS, faq_vectors):
        score = cosine_similarity(qv, vector)
        scored.append((score, text))

    scored.sort(reverse=True, key=lambda x: x[0])
    return scored[:top_k]

for score, text in semantic_search(
    "Can I work from home?"
):
    print(round(score, 4), text)
```

Step 5 - Add a Threshold

```
def search_with_threshold(query, threshold=0.60):
    results = semantic_search(query, top_k=5)
    return [
        (score, text)
        for score, text in results
        if score >= threshold
    ]
```

Teaching Warning

The threshold value above is only a classroom example. Real similarity distributions vary by embedding model, data, query type, normalization, and search implementation. Tune thresholds with an evaluation set.

14. Evaluation & Failure Modes

What Should We Evaluate?

Metric / Check	Question
Recall@K	Did the correct chunk appear somewhere in the top K?
Precision@K	How many of the returned chunks are actually relevant?
MRR intuition	How high was the first correct result ranked?
Abstention	Does the system reject weak evidence?
Latency	How long does retrieval take?
Filter correctness	Did authorization / metadata rules work?

Create a Retrieval Test Set

Question: "Can I work from home?"
Expected chunk: remote_work_policy_02

Question: "How long do I have to file expenses?"
Expected chunk: expenses_30_day_rule

Question: "Where do I reset my password?"
Expected chunk: password_reset_portal

Common Failure Modes

- **Wrong embedding model:** document and query vectors are incompatible or poorly suited.
- **Bad chunks:** one chunk contains too many unrelated topics.
- **Low recall:** relevant chunk never enters the candidate set.
- **Low precision:** many noisy chunks crowd the context.
- **Wrong metadata filter:** useful documents are excluded.
- **No threshold:** system always returns something, even when nothing is relevant.
- **Identifier problem:** semantic search is weak for exact IDs without lexical support.
- **Stale index:** source documents changed but vectors were not refreshed.

15. Business Use Cases

Embeddings are a general similarity technology, not only a RAG feature.

Business Use Case	Embedding / Vector Search Role
Internal knowledge assistant	Find policy/document chunks related to employee question
Customer support	Match ticket to similar solved cases or knowledge articles
Sales	Find similar customer accounts, use cases, or proposals
Recruiting	Match role requirements to candidate experience summaries
Ecommerce	Semantic product search and related-product recommendations
Legal operations	Retrieve clauses with similar meaning across contracts
Fraud / anomaly review	Identify items that are unusually dissimilar from normal patterns
Content organization	Cluster articles or tickets by semantic topic

Outsourcing Job Perspective

When a client says, “Our chatbot cannot find the right documents,” an AI Automation Architect should investigate the retrieval stack instead of immediately rewriting the prompt.

Check:

1. Source documents
2. Parsing
3. Chunk boundaries
4. Embedding model
5. Metadata
6. Search mode
7. Top-K
8. Threshold
9. Ranking / reranking
10. Evaluation dataset

Professional Mindset

A RAG answer can fail because of generation, but it can also fail much earlier because the retrieval system selected the wrong evidence.

16. Exercise, Quiz & Homework

Class Exercise A - Keyword or Semantic?

Query	Best starting approach
"invoice INV-99381"	Keyword / exact filter
"How do I get my expenses paid back?"	Semantic
"error E_CONN_104"	Keyword / exact token
"Can I work from home?"	Semantic
Product SKU + natural language description	Hybrid

Class Exercise B - Rank the Results

Question: "Can unused vacation days move into next year?"

Candidate A: "Unused vacation days may carry over up to 5 days."
Candidate B: "New employees receive 15 vacation days."
Candidate C: "Expense claims must be submitted within 30 days."

Expected ranking: A > B > C

10-Question Quiz

1. What is an embedding?

Answer: A numerical representation designed to preserve useful relationships such as semantic similarity.

2. What does semantic search compare?

Answer: The query representation with stored representations to find meaning-related items.

3. What does top-K mean?

Answer: Return the K highest-ranked retrieval candidates.

4. Why use a score threshold?

Answer: To reject weak candidates and support abstention.

5. What is metadata filtering for?

Answer: Restrict search by structured attributes such as department, region, date, or access scope.

6. What is ANN?

Answer: Approximate nearest-neighbor search for fast vector retrieval at scale.

7. What is hybrid search?

Answer: Combining semantic/vector search with lexical/keyword evidence.

8. Why can a RAG answer fail even with a good LLM?

Answer: The retriever may have selected weak or wrong evidence.

9. What should store current order status: memory or API?

Answer: API / authoritative business system.

10. Should one similarity threshold be copied to every project?

Answer: No. Tune using the model, data, query distribution, and eval set.

Homework

- Build the five-item FAQ semantic search example and test at least 10 differently worded queries.
- For each query, record the top-3 results and whether the expected answer appears in the top 1, top 3, or not at all.
- Experiment with top-K values of 1, 3, and 5. Describe the precision/noise tradeoff.
- Add metadata fields such as department and year, then design a filtering rule.
- Create three exact-identifier queries where semantic search is insufficient and explain why hybrid search would help.
- Design a 20-question retrieval evaluation dataset for an employee handbook.
- Create a one-page architecture diagram: documents -> chunks -> embeddings -> vector store -> query embedding -> search -> LLM.

17. Interview Questions & Day 18 Preview

Interview Questions

What is an embedding?

A vector representation where useful relationships between items can be measured mathematically.

What is cosine similarity?

A measure of how closely two vectors point in the same direction.

Semantic search vs keyword search?

Semantic search focuses on meaning; keyword search focuses on exact lexical overlap.

What is a vector database/store?

A system that stores vectorized items and supports efficient similarity retrieval, often with metadata.

What is top-K?

The number of highest-ranked candidates returned from retrieval.

Why use metadata filters?

To narrow retrieval to relevant and authorized subsets.

What is approximate nearest-neighbor search?

A scalable method that finds very close vectors without exhaustively comparing every vector.

Why use hybrid search?

To combine semantic meaning with exact terms, identifiers, and keyword evidence.

How do you evaluate retrieval?

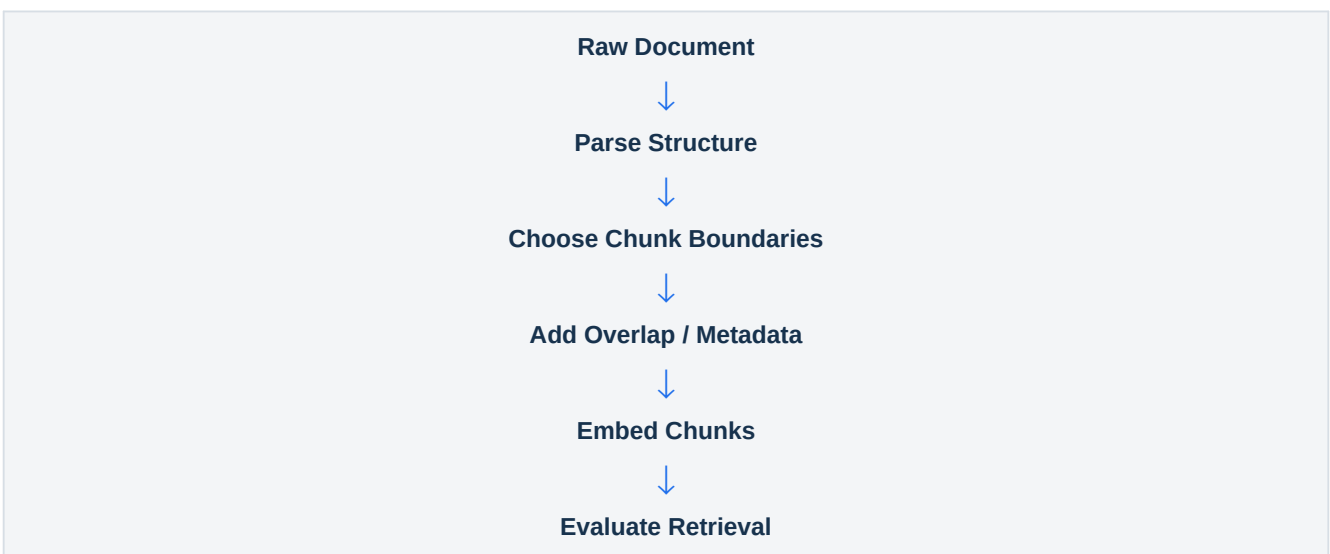
Use labeled queries and metrics/checks such as recall@K, precision@K, ranking quality, thresholds, and latency.

Where do embeddings fit in RAG?

They represent chunks and queries so the retriever can identify semantically relevant evidence before generation.

Day 18 Preview - Chunking Documents in Depth

Day 18 will focus on the part of RAG that determines **what exactly gets embedded**. Students will learn fixed-size chunking, recursive splitting, semantic boundaries, overlap, section-aware chunking, tables, metadata, parent-child retrieval, and how chunk size changes recall and precision.



Day 17 Final Takeaway

Embeddings make semantic similarity computable. Vector search turns that representation into retrieval. RAG succeeds when the system embeds the right units, searches them well, and sends only strong evidence to the LLM.

18. References

Current OpenAI-specific implementation details in this lecture were verified on August 18, 2026 using official OpenAI developer documentation.

Official OpenAI Sources

- OpenAI Developers - Vector embeddings: <https://developers.openai.com/api/docs/guides/embeddings>
- OpenAI Developers - Retrieval: <https://developers.openai.com/api/docs/guides/retrieval>
- OpenAI Developers - File search: <https://developers.openai.com/api/docs/guides/tools-file-search>
- OpenAI API Reference - Search vector store:
https://developers.openai.com/api/reference/resources/vector_stores/methods/search

Concepts to Continue Researching

- Approximate nearest-neighbor algorithms such as HNSW.
- Hybrid lexical + dense retrieval.
- Cross-encoder or LLM-based reranking.
- Retrieval evaluation: Recall@K, Precision@K, MRR, nDCG.
- Vector database choices: pgvector, Qdrant, Pinecone, Weaviate, Elasticsearch/OpenSearch vector search.

End of Day 17 Lecture